

Connecting On-Premises Apache Kafka to Microsoft Fabric

Network architecture, ingestion options, and decision guide

September 2026

Abstract

This whitepaper describes the main architectures for moving events from an on-premises Apache Kafka cluster into Microsoft Fabric. It compares Eventstream connector virtual network injection, Kafka Connect to Eventhouse, the Eventstream Kafka endpoint, Azure Event Hubs as an intermediary, and a relay for networks limited to outbound HTTPS on port 443.

The document covers connection direction, ports, authentication, DNS, certificates, operational ownership, selection criteria, and a practical proof-of-concept plan.

Contents

1	Executive summary	2
1.1	Document color key	3
2	Start with connection direction	4
3	Kafka network behavior that matters	4
4	Architecture options	5
5	Option A: Eventstream Apache Kafka connector with VNet injection	6
5.1	Architecture	6
5.2	How it works	6
5.3	Azure and Fabric prerequisites	6
5.4	Subnet sizing	7
5.5	Network flows	7
5.6	Authentication and TLS	7
5.7	DNS	8
5.8	Limitations and operating notes	8
5.9	Variant: Connector IP Allowlist	8
5.10	When to use this option	8
6	Option B: Kafka Connect to Eventhouse over HTTPS	9
6.1	Architecture	9
6.2	Advantages	9
6.3	Operating constraints	9
6.4	Identity	10
6.5	Access through OneLake	10
6.6	When to use this option	10
7	Option C: push Kafka data to an Eventstream custom endpoint	11
7.1	Advantages	11
7.2	Constraints	11
7.3	When to use this option	11
8	Option D: Azure Event Hubs as an intermediary	12
8.1	Architecture	12
8.2	Advantages	12
8.3	Constraints	12

8.4	Port 443 variant	13
8.5	When to use this option	13
9	Option E: on-premises relay to Eventstream over HTTPS 443	14
9.1	Advantages	14
9.2	Constraints	14
10	Option comparison	15
11	Decision tree	17
12	Diagnosing “403” and “443”	17
13	Common security baseline	18
13.1	Network	18
13.2	Identity and secrets	18
13.3	TLS and certificates	19
13.4	Reliability	19
14	Implementation approach	19
14.1	Step 1: collect the facts	19
14.2	Step 2: shortlist two options	19
14.3	Step 3: prove the network before configuring Fabric	19
14.4	Step 4: run a functional proof of concept	20
14.5	Step 5: prepare production operations	20
15	Default recommendation	20
16	Questions to answer before selection	21
17	Product status and caveats	21
18	References	22
18.1	Microsoft Fabric Eventstream	22
18.2	Eventhouse and Kafka Connect	23
18.3	Azure Event Hubs	23

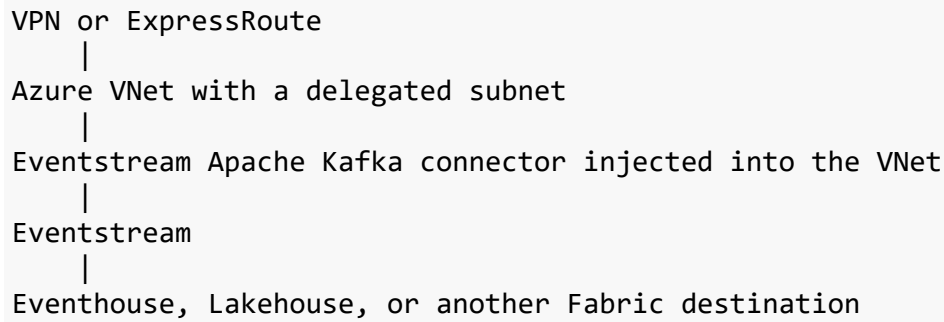
Scope. This document covers Apache Kafka running on-premises or in another private network. It is independent of any customer context and contains no organization names.

Service status. Product availability and limitations were checked on September 11, 2026. Microsoft can change them. Review the linked documentation before a production deployment.

1 Executive summary

When Microsoft Fabric must read from a private Kafka cluster, the native architecture is:

On-premises Kafka
|



This option has been generally available since July 2026. It keeps Eventstream transformations and routing, but it has one network requirement that often decides the outcome: the Fabric connector initiates connections to Kafka. The on-premises network must accept traffic from the delegated Azure subnet to every broker listener returned in Kafka metadata.

A policy that permits only outbound HTTPS on port 443 changes the shortlist:

Primary requirement	Recommended option
Keep Eventstream and allow Fabric to reach the brokers	Eventstream connector VNet injection
Use outbound HTTPS 443 only, without custom code	Kafka Connect with the Fabric sink to Eventhouse
Keep Eventstream and allow Kafka TLS on port 9093	Kafka protocol on an Eventstream custom endpoint
Add an Azure buffer and a separate network boundary	Azure Event Hubs between Kafka and Eventstream
Keep Eventstream while limiting outbound traffic to 443	On-premises relay to the custom endpoint over AMQP WebSockets or HTTPS

1.1 Document color key

- A** Private pull through a VNet
- B** HTTPS 443 push to Eventhouse
- C** Kafka TLS 9093 push to Eventstream
- D** Azure Event Hubs as an intermediary
- E** Port 443 relay to Eventstream

Clarify the reported “403 / HTTPS” issue before choosing an architecture. 403 is an HTTP status, not a port. It can point to a proxy denial, an access rule, an unauthorized identity, or TLS inspection. A “port 443 only” policy is different. It rules out native Kafka paths on port 9093 even though those connections are encrypted.

2 Start with connection direction

The first design question is which side opens the connection.

Model	Initiator	Network consequence
Eventstream pull	Fabric connects to Kafka	The on-premises network accepts connections from the Azure VNet to the brokers
Kafka Connect push	A worker near Kafka connects to Fabric	No new inbound connection to the on-premises network
Push to a custom endpoint	A producer or replication tool connects to Eventstream	The on-premises network needs an outbound path to Fabric
Push to Event Hubs	A producer or relay connects to Azure Event Hubs	Event Hubs then decouples the source from Fabric

Eventstream documentation classifies traffic by the connection initiator. An Apache Kafka source configured in Eventstream is outbound from Fabric because Eventstream retrieves events from Kafka. A custom endpoint is inbound because an external application pushes events into Fabric.

3 Kafka network behavior that matters

A Kafka client does not stay connected only to the bootstrap server. It uses that server to retrieve cluster metadata, then connects to the brokers that own the partitions. Every address returned through `advertised.listeners` must be resolvable and reachable from the client network.

For a hybrid connection, verify:

- every broker FQDN or IP address returned in metadata;
- the actual listener ports instead of assuming 9092 or 9093;
- DNS resolution from Azure;
- forward and return routing between the Azure subnet and the on-premises networks;
- Kafka ACLs for the topics and consumer group;
- the certificate chain presented by each broker.

Allowing only the bootstrap server is not enough when metadata points the client to other brokers.

4 Architecture options

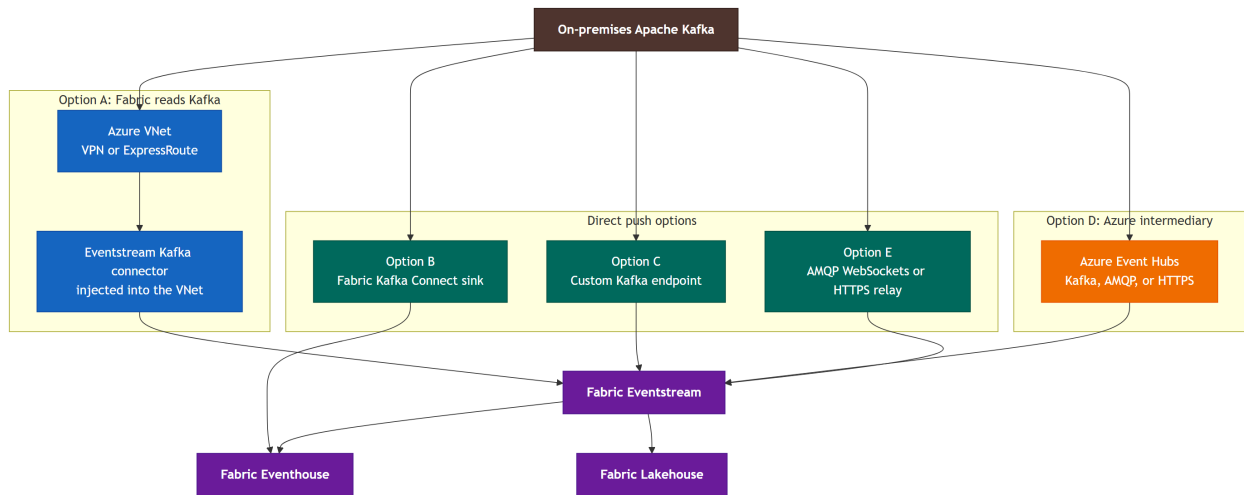


Figure 1: Architecture 1

OPTION A

Private pull through a VNet

5 Option A: Eventstream Apache Kafka connector with VNet injection

5.1 Architecture

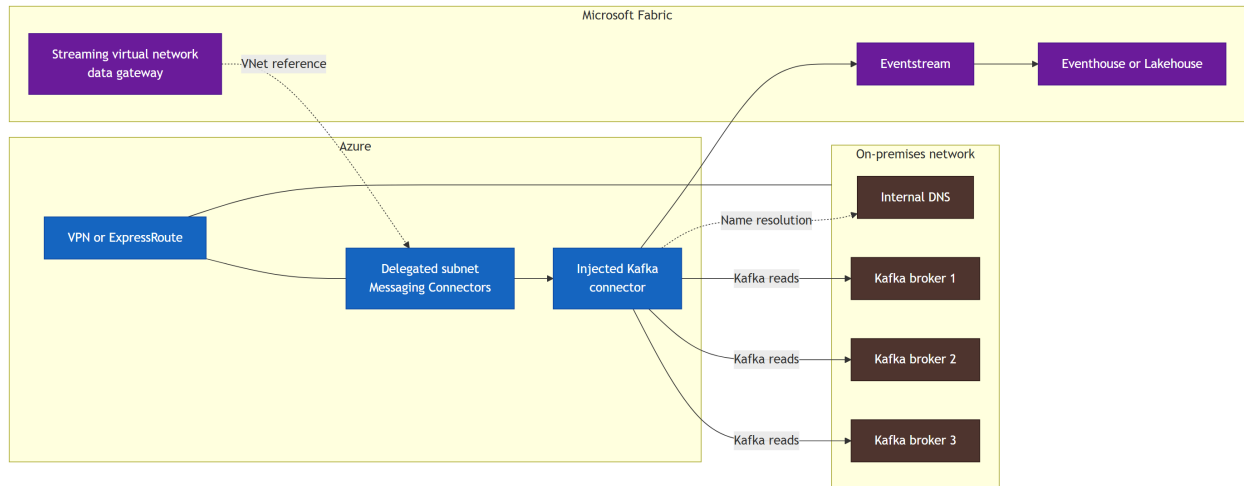


Figure 2: Architecture 2

5.2 How it works

Fabric creates a streaming connector instance in an Azure subnet prepared by the organization. The connector reaches Kafka through VPN or ExpressRoute. The streaming virtual network data gateway does not deploy a gateway cluster. It stores the VNet and subnet reference that Eventstream uses for connector injection.

This is the closest fit with Eventstream. Transformations, filtering, routing to several destinations, and Real-Time Intelligence operations stay in Fabric.

5.3 Azure and Fabric prerequisites

Microsoft documents the following setup:

1. Register the `Microsoft.MessagingConnectors` resource provider in the subscription that hosts the VNet.
2. Create or reuse an Azure VNet in the same region as the eventstream.
3. Avoid address overlap with `10.240.0.0/16` and `10.224.0.0/12`.
4. Prepare a dedicated subnet.
5. Delegate it to `Microsoft.MessagingConnectors/connectors`, shown as **Messaging Connectors** in the Azure portal.
6. Use at least a /27 with at least 16 available addresses.

7. Connect the VNet to the Kafka network through VPN or ExpressRoute.
8. Enable the Fabric workspace identity.
9. Grant that identity the Azure **Network Contributor** role on the VNet.
10. Create the streaming virtual network data gateway, then create a connection marked [vNet].

Microsoft recommends a new subnet. If an existing subnet is reused, it must not contain Private Endpoints, Load Balancers, Application Gateways, virtual machines, Virtual Machine Scale Sets, or network interfaces.

5.4 Subnet sizing

Azure reserves 15 addresses in this subnet for the service. Each connector uses at least one address and can scale up to the source partition count.

The documented example is:

```
15 reserved addresses
+ 10 partitions x 2 Kafka connectors
= up to 35 addresses required
```

The example needs more addresses than a /27 provides and normally calls for a /26. Size the subnet for the current connectors, their partition counts, and planned growth. A /27 is only the technical minimum.

5.5 Network flows

The connector initiates traffic from the delegated subnet to Kafka. Network rules must allow:

- the Kafka listener ports advertised by the brokers;
- the required DNS queries;
- return traffic through VPN or ExpressRoute;
- Azure Key Vault access when it stores certificates;
- any Azure and Fabric service egress required by the organization's policy.

From the on-premises firewall's point of view, these are inbound connections from the Azure VNet to the brokers. This is often the disputed part of the design even though the path is private.

5.6 Authentication and TLS

The Eventstream Apache Kafka connector documents:

- SASL_SSL with PLAIN, SCRAM-SHA-256, or SCRAM-SHA-512;
- SSL with mutual TLS;
- a public trusted certificate authority, or an internal CA configured through the TLS/mTLS settings;
- PEM certificates stored in Azure Key Vault;
- a server certificate whose SAN includes the names or addresses used to reach the brokers.

For a private source, connect the Key Vault that holds the certificates to the VNet used by the streaming virtual network data gateway, for example through a Private Endpoint. The person who configures the source and previews data also needs the required Key Vault permissions.

5.7 DNS

The connector must resolve every broker name returned by Kafka. Microsoft recommends proving that a virtual machine in the VNet can reach the source before Eventstream is configured.

The injected connector does not directly support a custom DNS server. Microsoft documents three workarounds:

- link an Azure Private DNS zone to the VNet and create the required A records;
- use Azure DNS Private Resolver to forward queries to the on-premises DNS service;
- use a private IP address when the connector and certificate rules do not require an FQDN.

Test every broker FQDN returned in metadata, not just the bootstrap server.

5.8 Limitations and operating notes

- The connection test is disabled when the connection uses a streaming virtual network data gateway.
- Source data preview is unavailable for private-network sources. Validate flow through source status, downstream nodes, and the destination.
- When preview is available for a public Kafka source, it supports JSON messages only.
- Private connectivity does not repair an incorrect `advertised.listeners` configuration.
- Delivery depends on coordination across the Fabric, Azure networking, Kafka, DNS, and PKI teams.

5.9 Variant: Connector IP Allowlist

If the organization cannot prepare a VNet, Microsoft documents a public-network variant. Each regional streaming connector has a single outbound IP address. The source must have a publicly resolvable address, and its firewall must allow that IP.

This variant:

- avoids VPN and ExpressRoute;
- crosses the public network;
- exposes the source through a public address protected by an allowlist;
- requires a request through the [Eventstream Streaming Connector IP allow list Request](#) form.

Use it only when the security policy accepts controlled public exposure.

5.10 When to use this option

Choose option A when:

- Eventstream must handle transformations or routing;
- VPN or ExpressRoute already exists or can be established;
- traffic from the Azure subnet to every broker is allowed;
- the network team can operate hybrid DNS and return routing.

Reject this option when policy forbids all connections initiated from Azure toward the on-premises network.

OPTION B

HTTPS 443 push to Eventhouse

6 Option B: Kafka Connect to Eventhouse over HTTPS

6.1 Architecture

```
On-premises Kafka
|
Kafka Connect in distributed mode
|
Microsoft Fabric sink
|
HTTPS 443
|
Eventhouse
|
OneLake availability, Lakehouse, Warehouse, Notebook, or Power BI
```

Current Microsoft Fabric guidance points to the Microsoft Azure Data Explorer Kafka Connect Kusto sink for Eventhouse. Kafka Connect workers run in the organization's chosen environment, preferably close to the cluster. They read Kafka locally, then call the Eventhouse ingestion and query endpoints over HTTPS.

A separate `microsoft/kafka-sink-ms-fabric` repository also exists, but Microsoft does not document it as a replacement for the Learn-linked Kusto sink. Select and pin one connector implementation during the proof of concept.

6.2 Advantages

- The on-premises environment initiates the connection.
- A public endpoint does not require VPN or ExpressRoute.
- Eventhouse ingestion and query endpoints use HTTPS URLs.
- The connector handles JSON, CSV, and Avro, topic-to-table mappings, retries, and dead-letter queues.
- Managed streaming ingestion can target subsecond latency when it is enabled and sized correctly.
- The connector documents `proxy.host` and `proxy.port`.

6.3 Operating constraints

- Production requires Kafka Connect in distributed mode.
- Queued ingestion is the default. Managed streaming is optional and can fall back to queued ingestion after retries.

- Queued ingestion uses temporary Azure Blob, Queue, and sometimes Table Storage endpoints in addition to the Eventhouse ingestion and query URLs. All use HTTPS 443, but a strict proxy must allow the service-issued Storage FQDNs.
- Delivery is **at least once**, so downstream processing must tolerate duplicates.
- Teams must administer tables, mappings, ingestion policies, and dead-letter queues.
- The documented proxy settings cover host and port. The connector does not document proxy authentication. Test it with the actual corporate proxy rather than assuming support.
- Strict Fabric Private Link needs a separate proof of concept because queued ingestion is unsupported and managed streaming can fall back to it.

6.4 Identity

The connector implementation includes three authentication strategies:

- a Microsoft Entra application with tenant ID, application ID, and secret;
- managed identity when the worker runs in a compatible Azure environment;
- workload identity in a platform that supports it.

For a fully on-premises deployment, a Microsoft Entra application is the clearest documented choice. Store its secret in the Kafka Connect platform's secret provider, not in plain text configuration. Managed identity depends on an Azure host with IMDS, while workload identity lacks an Eventhouse-specific deployment guide.

6.5 Access through OneLake

Enabling OneLake availability on the KQL database creates a read-only Delta representation for other Fabric engines. A Lakehouse can access it directly or through a shortcut.

This is not an immediate replacement for an Eventstream Lakehouse destination:

- the default write delay can reach three hours when files have not reached an efficient size;
- `TargetLatencyInMinutes` can be set between 5 minutes and 3 hours;
- shorter delays can create many small files;
- some operations are blocked while the feature is enabled, including table rename, column type changes, data deletion or purge, and row-level security.

If Eventhouse is the main analytics target, this option is straightforward. If a Delta table must appear in a Lakehouse within seconds, measure the actual latency before selecting it.

6.6 When to use this option

Choose option B when:

- policy permits outbound HTTPS 443 only;
- Eventhouse is an acceptable target;
- the organization already operates Kafka Connect or is prepared to do so;
- transformations can run before ingestion or inside Eventhouse.

For a tightly filtered network with no inbound path to Kafka, this is often the simplest starting point.

OPTION C

Kafka TLS 9093 push to Eventstream

7 Option C: push Kafka data to an Eventstream custom endpoint

An Eventstream custom endpoint exposes connection details compatible with Event Hubs, AMQP, and Kafka. Microsoft explicitly documents ordinary Kafka producers and consumers against this endpoint. Other Kafka tooling needs separate validation.

The documented Kafka configuration uses:

```
bootstrap.servers=<endpoint-provided-by-eventstream>
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule
↪  required username="$ConnectionString" password="<connection-string>;
```

Azure Event Hubs uses TCP 9093 for its Kafka protocol, and the Eventstream custom endpoint uses that compatibility layer. A policy limited to TCP 443 cannot use this Kafka path as-is.

7.1 Advantages

- The on-premises environment initiates the connection.
- Eventstream remains available for transformation and routing.
- The destination is one stable FQDN rather than every source-cluster broker.
- A public endpoint does not require VPN or ExpressRoute.

7.2 Constraints

- The firewall must allow outbound TCP 9093 to the *.servicebus.windows.net endpoint.
- Event Hubs implements the Kafka protocol but is not a complete Kafka broker. Test the replication tool and the APIs it uses.
- SAS secrets or configured identities require controlled storage and rotation.
- Microsoft does not document Kafka Connect or MirrorMaker 2 support for the Eventstream custom endpoint. Treat any such integration as an unverified proof of concept.
- Fabric Tenant and Workspace Private Links do not support custom endpoint sources or destinations.

7.3 When to use this option

Choose option C when Eventstream is required, the connection must start on-premises, and security permits TCP 9093 to a specific Azure endpoint.

OPTION D

Azure Event Hubs as an intermediary

8 Option D: Azure Event Hubs as an intermediary

8.1 Architecture

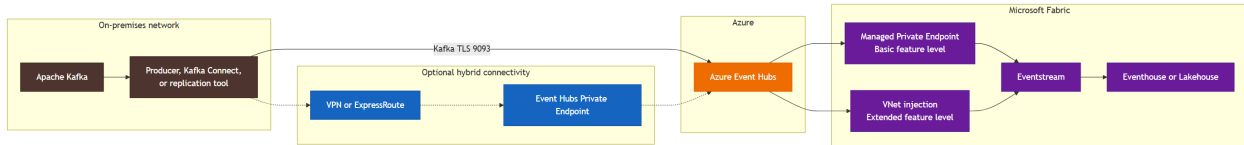


Figure 3: Architecture 3

Azure Event Hubs provides a Kafka endpoint in its Standard, Premium, and Dedicated tiers. Microsoft documents Kafka producers, Kafka Connect, MirrorMaker, and MirrorMaker 2 for this endpoint. Encrypted Kafka traffic uses TCP 9093.

Event Hubs can be exposed through:

- its public endpoint with firewall rules;
- a Private Endpoint reached from on-premises through VPN or ExpressRoute;
- both paths when policy permits them.

Eventstream can then read Event Hubs through:

- a Managed Private Endpoint at the Basic feature level;
- connector VNet injection at the Extended feature level.

Both Fabric network features are generally available.

Using the Fabric workspace identity as the Event Hubs source authentication method remains in preview. This is separate from the workspace identity required to grant Network Contributor for connector VNet injection.

8.2 Advantages

- Event Hubs absorbs temporary interruptions between Azure and Fabric.
- The Azure network boundary has its own lifecycle, separate from the eventstream.
- The namespace exposes one stable endpoint.
- Several consumers can read the same stream.
- Azure retention, Capture, metrics, and access controls are available.

8.3 Constraints

- The additional Azure service needs capacity planning, security, monitoring, and a budget.
- Event Hubs for Kafka is not a complete Kafka cluster.

- The native Kafka path requires TCP 9093, including through a Private Endpoint.
- Event Hubs Private Endpoint is unavailable in the Event Hubs Basic tier.
- Private connectivity requires DNS for `privatelink.servicebus.windows.net` and the correct hybrid route.

8.4 Port 443 variant

Event Hubs accepts producers over HTTPS 443 and AMQP WebSockets on 443. This requires a relay or application that consumes Kafka and publishes through the Event Hubs SDK or HTTPS API. MirrorMaker 2 does not become a port 443 client by changing its destination port.

8.5 When to use this option

Choose option D when the architecture needs an Azure buffer, a separate network boundary, or a Private Endpoint managed independently from the Fabric workspace.

OPTION E

Port 443 relay to Eventstream

9 Option E: on-premises relay to Eventstream over HTTPS 443

This option covers the case where Eventstream is required but the firewall permits only outbound TCP 443.

```
On-premises Kafka
|
Local relay service
|
Event Hubs SDK with AMQP WebSockets on port 443
or HTTPS POST requests
|
Eventstream custom endpoint
|
Fabric transformations and destinations
```

The Eventstream custom endpoint provides an Event Hubs-format connection string. Event Hubs SDKs can use AMQP WebSockets, and the Event Hubs FAQ confirms that this transport can run entirely on TCP 443. The HTTPS API can also send events, but it cannot receive them.

9.1 Advantages

- The on-premises environment initiates the connection.
- Eventstream stays in the architecture.
- Outbound traffic uses TCP 443.
- The relay can implement the organization's proxy policy, rate control, and logging requirements.

9.2 Constraints

- The organization must develop or maintain a relay.
- The relay owns Kafka offsets, retries, duplicate handling, and dead-letter processing.
- Test TLS inspection and authenticated proxy behavior with the selected SDK.
- Measure throughput and batch sizes under representative load.
- The Eventstream custom endpoint remains public because Fabric Private Link does not support custom endpoint sources or destinations.

This option costs more to operate than Kafka Connect to Eventhouse. It is justified when Eventstream transformations are required and TCP 9093 is not allowed.

10 Option comparison

Option	Initiator	Main outbound port from on-premises	VPN or ExpressRoute	Eventstream	Component to operate
A. VNet injection	Fabric to Kafka	Private Kafka listener port	Yes	Yes	Hybrid network and Eventstream configuration
A2. IP allowlist	Fabric to public Kafka	Publicly exposed Kafka listener HTTPS 443	No	Yes	Controlled public exposure
B. Kafka Connect to Eventhouse	On-premises to Fabric		No	No	Kafka Connect cluster
C. Kafka custom endpoint	On-premises to Fabric	Kafka TLS 9093	No	Yes	Kafka producer or custom integration
D. Event Hubs intermediary	On-premises to Azure	9093, or 443 with a relay	Optional, required for Private Endpoint	Yes	Event Hubs and producer
E. Relay to Eventstream	On-premises to Fabric	HTTPS 443	No	Yes	Relay service

Criterion	A	B	C	D	E
No connection initiated toward on-premises	No	Yes	Yes	Yes	Yes
Compatible with a port 443 only policy	No	Yes	No	Yes with a relay	Yes
Eventstream transformations	Yes	No	Yes	Yes	Yes

Criterion	A	B	C	D	E
Buffer independent from Fabric	No	No	No	Yes	No
Little custom code	Yes	Yes	Yes if the tool is compatible	Yes with Kafka 9093	No
Hybrid network complexity	High	Low	Low	Medium to high	Low
Application operating effort	Low	Medium	Medium	Medium	High

11 Decision tree

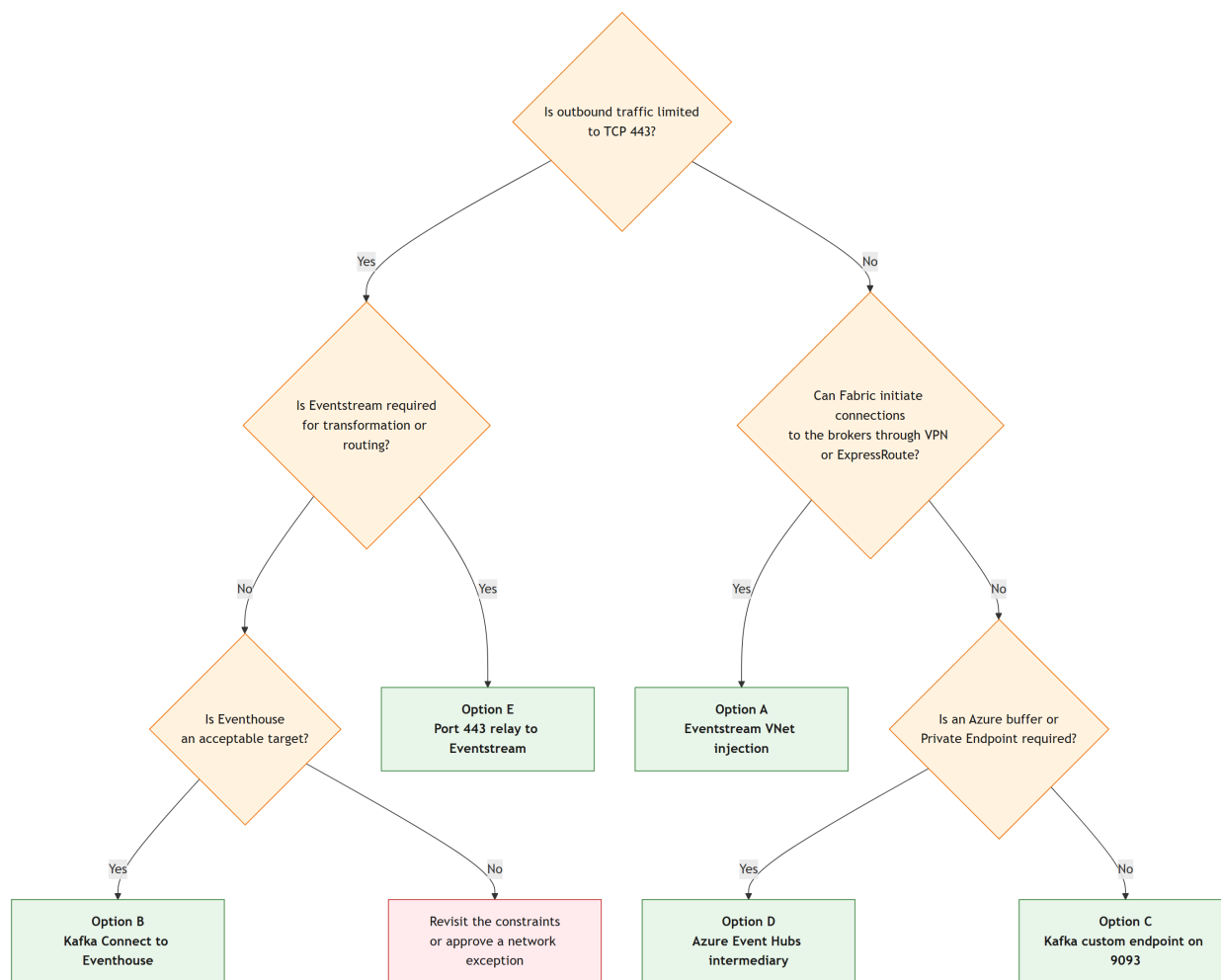


Figure 4: Architecture 4

12 Diagnosing "403" and "443"

Identify the actual restriction before selecting an option.

Observation	Likely meaning	Check
The proxy returns HTTP 403 Forbidden	The HTTPS request reached the proxy, but a rule rejected the FQDN, CONNECT method, identity, or destination	Read the proxy log and the rule that produced the denial
The connection to 9093 times out	The firewall blocks Kafka TLS or the network path is incomplete	Check DNS, routing, and Test-NetConnection <fqdn> -Port 9093

Observation	Likely meaning	Check
TLS fails on port 443	TLS inspection, an untrusted CA, SNI handling, or protocol version	Capture the certificate chain from the production runtime
Eventstream cannot reach Kafka	An advertised listener does not resolve, a broker port is blocked, return routing is missing, or Kafka ACLs reject the client	Read Kafka metadata from a VM in the VNet
Fabric does not offer a connection test	Expected with the streaming virtual network data gateway	Publish, then inspect source state and eventstream data preview
Preview is empty	Non-JSON data, consumer-group permissions, Key Vault access, or no current events	Read the topic with a reference consumer and check permissions

Ask the network and security teams:

1. Does "403" mean an HTTP response from a proxy, or was it shorthand for a port 443 only rule?
2. Are connections from an Azure VNet to the on-premises Kafka brokers forbidden?
3. Can the firewall allow TCP 9093 to one specific Azure FQDN?
4. Does the proxy permit AMQP over WebSockets, or only conventional HTTP requests?
5. Does TLS inspection replace the certificate presented to the client?
6. Are the required Fabric and Microsoft Entra endpoints allowed by FQDN?

13 Common security baseline

13.1 Network

- Restrict rules to the CIDRs, FQDNs, and ports actually used.
- Do not expose Kafka brokers to the internet to work around a routing problem.
- For option A, keep the delegated subnet separate from Private Endpoints and virtual machines.
- For private Event Hubs, use a Private Endpoint and disable public access when policy requires it.
- Log denials in the firewall, proxy, NSGs, and VPN platform.

13.2 Identity and secrets

- Prefer Microsoft Entra identities over static secrets when the runtime supports them.
- Store Kafka, SAS, and Microsoft Entra secrets in a vault or secret provider.
- Separate ingestion, administration, and read identities.
- Grant only the required rights on topics, consumer groups, tables, and databases.
- Define a rotation procedure that does not interrupt the stream.

13.3 TLS and certificates

- Use SASL_SSL or mTLS for Kafka.
- Match certificate SANs to the advertised broker names.
- Import complete certificate chains and required private keys in the expected format.
- Test the internal CA from the connector runtime, not only from an administrator workstation.
- Document how TLS inspection affects Kafka, AMQP WebSockets, and HTTPS.

13.4 Reliability

- Design downstream processing for at-least-once delivery.
- Define an idempotency key that Eventhouse or the transformation layer can use.
- Provide a dead-letter path and a replay procedure.
- Monitor Kafka lag, offsets, retries, ingestion failures, and throughput.
- Test an outage that lasts longer than local buffers.

14 Implementation approach

14.1 Step 1: collect the facts

Gather the following information:

Area	Required information
Kafka	Distribution, version, broker count, topics, partitions, throughput, message size, retention
Kafka security	SASL, SCRAM, mTLS, optional Kerberos, CA, ACLs, schema registry
Network	CIDRs, DNS, ports, VPN or ExpressRoute, proxy, TLS inspection, inbound and outbound rules
Fabric	Capacity region, workspace, Eventstream feature level, Eventhouse or Lakehouse destination
Operations	RTO, RPO, target latency, replay, monitoring, on-call ownership

14.2 Step 2: shortlist two options

Run a proof of concept only for options that satisfy the real network rules:

- options A and B when the choice is between a private pull path and a port 443 push path;
- options B and E when outbound traffic is strictly limited to 443;
- options C and D when 9093 is allowed and Eventstream must remain in the design.

14.3 Step 3: prove the network before configuring Fabric

For option A, deploy a test VM in a nondelegated subnet of the same VNet and run:

```
Resolve-DnsName kafka-broker-1.example.internal  
Test-NetConnection kafka-broker-1.example.internal -Port <listener-port>
```

Then use a Kafka client to read cluster metadata. Every returned broker address must be reachable from Azure.

For options C and D:

```
Resolve-DnsName <namespace>.servicebus.windows.net  
Test-NetConnection <namespace>.servicebus.windows.net -Port 9093
```

For options B and E:

```
Test-NetConnection <fabric-or-eventstream-endpoint> -Port 443
```

Run these tests from the server or container that will host Kafka Connect or the relay. A test from a user workstation does not prove the production path.

14.4 Step 4: run a functional proof of concept

The proof of concept should cover:

- several partitions and brokers;
- worker stop and restart;
- a temporary network outage;
- secret or certificate rotation;
- a malformed message and dead-letter handling;
- duplicate delivery and replay from an earlier offset;
- representative load;
- latency measurement at each boundary;
- access to data in the final Fabric destination.

14.5 Step 5: prepare production operations

Before production:

1. Automate Azure resources, role assignments, and Fabric configuration where APIs support them.
2. Make Kafka Connect workers or the relay highly available.
3. Alert on lag, authentication failures, network denials, and ingestion errors.
4. Write recovery, rotation, replay, and certificate-change procedures.
5. Test recovery after a complete hybrid-path outage.
6. Obtain approval from the network, security, Kafka, and Fabric owners.

15 Default recommendation

Choose based on the network constraint:

- If VPN or ExpressRoute exists and Fabric may connect to every broker, use option A.

- If the on-premises network permits only outbound HTTPS 443 and Eventhouse is suitable, use option B.
- If Eventstream is required and TCP 9093 is allowed, use option C.
- If the design needs an Azure buffer or separately managed Private Endpoint, use option D.
- If Eventstream is required and TCP 443 is the only allowed port, use option E.

In many tightly filtered environments, the practical first comparison is A versus B. Option A keeps more work inside Fabric. Option B is easier to pass through a restrictive firewall because every connection starts on-premises.

16 Questions to answer before selection

1. Does "403" mean an HTTP response, a proxy denial, or confusion with port 443?
2. Can an Azure VNet initiate connections to the Kafka network?
3. Is VPN or ExpressRoute already available in the relevant Azure region?
4. Which region hosts the Fabric capacity and workspace?
5. Which FQDNs and ports do the brokers advertise?
6. Does the cluster use SASL/SCRAM, mTLS, Kerberos, or an internal CA?
7. Can Azure DNS resolve the internal Kafka names?
8. Is the final target Eventhouse, Lakehouse, or several destinations?
9. Are Eventstream transformations required?
10. What end-to-end latency must the design meet?
11. Does the organization already operate Kafka Connect?
12. Is an Azure buffer useful for replay and decoupling?

17 Product status and caveats

Feature	Verified status	Caveat
Eventstream connector VNet injection	GA	Requires an Azure VNet, delegated subnet, and hybrid connectivity
Eventstream Managed Private Endpoint to Event Hubs or IoT Hub	GA	Azure source support is limited to Event Hubs and IoT Hub
Microsoft Azure Data Explorer Kafka Connect Kusto sink	Current Microsoft OSS release	Current Fabric Learn guidance for Kafka to Eventhouse; queued ingestion reaches temporary Storage endpoints
Separate Microsoft Fabric Kafka Connect sink repository	Microsoft project available	Microsoft does not document it as the replacement for the Learn-linked Kusto sink
Eventstream custom endpoint with Kafka protocol	Documented	Kafka TLS uses port 9093
OneLake availability for Eventhouse	Available	Adaptive Delta latency and restrictions on some table operations

Feature	Verified status	Caveat
Connector IP Allowlist	Available by request	Uses the public network and requires a publicly resolvable source
Private Links with an Eventstream custom endpoint	Not supported	Dedicated Fabric security documentation and the Eventstream support matrix both exclude custom endpoint sources and destinations
Event Hubs source authentication with Fabric workspace identity	Preview	The Eventstream network features themselves are GA
Eventhouse direct ingestion with Private Links	Not supported	Preprocessing mode is listed as supported

18 References

18.1 Microsoft Fabric Eventstream

Resource	Link
Choose the right Eventstream network security feature	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/choose-the-right-network-security-feature
VNet and on-premises support guide	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/streaming-connector-private-network-support-guide
VNet and on-premises support overview	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/streaming-connector-private-network-support-overview
Create and manage a streaming virtual network data gateway	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/create-manage-streaming-virtual-network-data-gateways
Add an Apache Kafka source	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/add-source-apache-kafka
Add a custom endpoint source	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/add-source-custom-app
Use the Eventstream Kafka endpoint	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/stream-consume-events-use-kafka-endpoint
Managed Private Endpoint for Eventstream	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/set-up-private-endpoint

Resource	Link
Tenant and Workspace Private Links	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-streams/set-up-tenant-workspace-private-links
Fabric Private Link support matrix	https://learn.microsoft.com/en-us/fabric/security/security-private-links-overview

18.2 Eventhouse and Kafka Connect

Resource	Link
Microsoft Azure Data Explorer Kafka Connect Kusto sink	https://github.com/Azure/kafka-sink-azure-kusto
Separate Microsoft Fabric Kafka Connect sink repository	https://github.com/microsoft/kafka-sink-ms-fabric
Ingest Kafka data into a Fabric KQL database	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/get-data-kafka
OneLake availability for Eventhouse	https://learn.microsoft.com/en-us/fabric/real-time-intelligence/event-house-onelake-availability

18.3 Azure Event Hubs

Resource	Link
Apache Kafka protocol support	https://learn.microsoft.com/en-us/azure/event-hubs/azure-event-hubs-apache-kafka-overview
Kafka migration and MirrorMaker guidance	https://learn.microsoft.com/en-us/azure/event-hubs/apache-kafka-migration-guide
Kafka Connect integration	https://learn.microsoft.com/en-us/azure/event-hubs/event-hubs-kafka-connect-tutorial
MirrorMaker 2 sample	https://github.com/Azure/azure-event-hubs-for-kafka/tree/master/tutorials/mirror-maker-2
Ports, HTTPS, and AMQP WebSockets	https://learn.microsoft.com/en-us/azure/event-hubs/event-hubs-faq
Event Hubs Private Endpoint	https://learn.microsoft.com/en-us/azure/event-hubs/private-link-service

Verified on September 11, 2026.